

SPÉCIFICATIONS TECHNIQUES DÉTAILLÉES : DONNÉES & INGESTION

RÉFÉRENCE : SPECS-DATA-INGEST-V1 PARENT : SAT_QHP_Vector_V1 DATE : 2026-01-12
STATUT : FROZEN

PARTIE A : DICTIONNAIRE DE DONNÉES ET SCHÉMAS DE PERSISTANCE (DDP)

Cette section définit la topologie de stockage physique et logique requise pour alimenter le Core Engine (Vectorbt Pro/Numba) avec une latence d'accès minimale.

1. ARCHITECTURE DE STOCKAGE (HDF5/ZARR)

Le système utilise une structure hiérarchique orientée colonnes pour maximiser la vitesse de lecture séquentielle ("slicing") requise par les opérations vectorielles.

- **Format Pivot :** HDF5 (.h5) avec compression `blosc:lz4` (Compromis Vitesse/Taux).
- **Structure de Groupe :** `/data/{exchange}/{symbol}/{timeframe}`
- **Mode d'Accès :** `Read-Only` pour le Core Engine, `Append-Only` pour l'Ingestion.

1.1. Schéma de Typage Strict (OHLCV)

Pour garantir l'absence de *type casting* coûteux lors du chargement en mémoire (RAM) vers les structures NumPy, les types disque doivent correspondre aux types natifs du processeur (x64).

Champ	Type Numpy	Type HDF5	Description	Contrainte d'Intégrité
open_time	int64	INT64	Timestamp UNIX (ms).	Clé Primaire, Unique, Monotone Croissant.
open	float64	FLOAT64	Prix d'ouverture.	> 0
high	float64	FLOAT64	Prix haut.	$\geq Open, \geq Low, \geq Close$
low	float64	FLOAT64	Prix bas.	$> 0, \leq High$
close	float64	FLOAT64	Prix de clôture.	> 0
volume	float64	FLOAT64	Volume base asset.	≥ 0
quote_vol	float64	FLOAT64	Volume quote asset.	≥ 0

trades	int32	INT32	Nombre de transactions.	≥ 0
--------	-------	-------	-------------------------	----------

2. PROTOCOLES D'INDEXATION ET D'ALIGNEMENT

L'alignement temporel est critique pour la vectorisation multicouche (ex: Overlay SMA 1H sur Chart 5m).

2.1. Indexation Temporelle Continue

Le stockage ne doit tolérer aucun "trou" (gap).

- **Règle** : Si absence de données (ex: maintenance exchange), les lignes temporelles manquantes doivent être générées.
- **Remplissage** : $\text{Open}=\text{Close}=\text{High}=\text{Low}=\text{Previous_Close}$, $\text{Volume}=0$.
- **Objectif** : Garantir que $\text{len}(\text{Array})$ est prévisible par la formule $\frac{T_{\text{end}}-T_{\text{start}}}{\text{Interval}}$.

2.2. Matrice de Rééchantillonnage Asymétrique (Broadcasting)

Lorsqu'une donnée de fréquence basse (F_{low} , ex: 1H) est projetée sur une fréquence haute (F_{high} , ex: 5m), le protocole suivant s'applique :

1. **Méthode Interdite** : `Backfill` (Biais de futur / Look-ahead bias).
2. **Méthode Mandatoire** : `Forward Fill` avec contrainte de clôture.
 - La valeur de `Close_1H` à 10:00 n'est connue qu'à 10:59:59.
 - Sur l'échelle 5m, cette valeur n'est disponible (visible) qu'à l'index 11:00.
 - Tout calcul à 10:30 doit utiliser la valeur `Close_1H` de 09:00 (valeur précédente validée).

PARTIE B : CONTRAT D'INTERFACE D'INGESTION (ICD) - BINANCE

Ce contrat spécifie les interactions entre le module `DataManager` et l'API Binance (Spot/Futures).

1. LOGIQUE DE SYNCHRONISATION DIFFÉRENTIELLE

L'ingestion ne doit jamais retélécharger l'historique complet sauf commande explicite (`--force-full`).

Algorithme de mise à jour (`update_routine`) :

1. **Lecture État Local** :
 - Ouvrir HDF5 : `/data/binance/BTCUSDT/5m` .
 - Extraire `last_timestamp` (T_{last}).
2. **Détermination Delta** :
 - $T_{\text{start_req}} = T_{\text{last}} + 1_Interval$.

- $T_{now} = \text{Current_Server_Time}$.
- Si $T_{start_req} > T_{now}$, Fin du processus (Up to date).

3. Boucle de Pagation (Chunking) :

- Tant que $T_{fetch} < T_{now}$:
 - GET /api/v3/klines avec `startTime = T_{fetch}` et `limit = 1000` .
 - Validation de la contiguïté des timestamps reçus.
 - Bufferisation en mémoire.
 - $T_{fetch} = \text{Dernier_TS_Reçu} + 1_Interval$.

4. Écriture Atomique :

- Append du buffer dans le Dataset HDF5.
- Mise à jour des métadonnées.

2. GESTION DES QUOTAS API (RATE LIMITS)

Le respect des poids (Weights) est impératif pour éviter le bannissement IP.

- **Endpoint :** GET /klines (Poids = 1).
- **Header Surveillé :** X-MBX-USED-WEIGHT-1M .
- **Seuil de Sécurité :** 80% du quota (ex: 960/1200 par minute).
- **Mécanisme de Freinage (Throttling) :**
 - Si `Used_Weight > Seuil` : Injection de `sleep(exponential_backoff)` .
 - Si HTTP 429 ou 418 reçu : Arrêt immédiat ("Kill Switch") pour `Retry-After` secondes.

3. TRAITEMENT DES FLUX TEMPS RÉEL (WEBSOCKETS)

Pour l'affichage "Live" et le déclenchement de signaux intra-barre.

- **Endpoint WS :** `wss://stream.binance.com:9443/ws`
- **Payload Subscription :**

```
{
  "method": "SUBSCRIBE",
  "params": ["btcusdt@kline_5m", "ethusdt@kline_5m"],
  "id": 1
}
```

- **Gestion de la Volatilité (kline_closed) :**
 - Le flux envoie des updates continus pour la bougie en cours (`x: false`).
 - Ces données vont dans le **Cache Volatile (RAM)** pour l'UI.

- Seule la clôture de bougie (`x: true`) déclenche l'écriture disque et le recalcul des indicateurs.

4. GESTION DES INTERRUPTIONS ET RÉSILIENCE

- **Heartbeat** : Ping applicatif toutes les 3 minutes pour maintenir la connexion WS active (`pong frame`).
- **Détection de Rupture** :
 - Si aucun message reçu depuis $2 \times \text{Intervalle}$, considérer la connexion comme "Stale".
 - Action : Reconnexion forcée + Appel API REST pour combler le "Gap" potentiel survenu